

COEP Technological University Pune

(A Unitary Public University of Govt. of Maharashtra)

School of Computational Sciences

Curriculum Structure All Four Years

B. Tech. Program in Computer Science and Engineering

List of Abbreviations

Abbreviation	Title
BS	Basic Science Course
ESC	Engineering Science Course
PCC	Programme Core Course (PCC)
PEC	Programme Elective Course (PEC)
OE/SE	Open/School Elective (OE/SE) other than particular program
MD M	Multidisciplinary Minor (MD M)
VSEC	Vocational and Skill Enhancement Course (VSEC)
HSMC	Humanities Social Science and Management
IKS	Indian Knowledge System (IKS)
VEC	Value Education Course (VEC)
RM	Research Methodology (RM)
--	Internship
--	Project
CEA	Community Engagement Activity (CEA)/Field Project
CCA	Co-curricular & Extracurricular Activities (CCA)

F.Y. B. Tech.
Computer Science and Engineering
[Level 4.5, UG Certificate] Semester -I

Sr. No.	Course Type	Course Code	Course Name	L	T	P	S	Cr	Evaluation Scheme (Weightages in %)				
									Theory			Laboratory	
									MSE	TA	ESE	ISE	ESE
01	BSC	<tbd>	Engineering Physics	2	0	2	1	3	30	20	50	CIE: 100	
02	BSC	<tbd>	Linear Algebra	2	1	0	1	3	30	10	60	--	--
03	BSC	<tbd>	Biology for Engineers	2	0	0	1	2	30	20	50	--	--
04	ESC	<tbd>	Basic Electrical & Electronics Engineering	3	0	2	1	4	30	20	50	CIE: 100	
05	ESC	<tbd>	Engineering Graphics and Design	1	0	4	1	3	CIE: 100			CIE: 100	
06	ESC	<tbd>	Computer Lab	0	0	2	0	1	--	--	--	CIE: 100	
07	AEC	<tbd>	IKS	2	0	0	0	2	CIE: 100			--	--
08	CCA	<tbd>	Liberal Learning -1	0	0	2	0	1	--	--	--	CIE: 100	
Total				12	01	12	05	19					

[Level 4.5, UG Certificate] Semester -II

Sr. No.	Course Type	Course Code	Course Name	L	T	P	S	Cr	Evaluation Scheme (Weightages in %)				
									Theory			Laboratory	
									MSE	TA	ESE	ISE	ESE
01	BSC	<tbd>	Probability and Statistics	2	0	2	1	3	30	20	50	CIE: 100	
02	BSC	<tbd>	Quantum Physics	2	0	2	1	3	30	20	50	CIE: 100	
03	ESC	<tbd>	Digital Logic Design	3	0	2	1	4	30	10	60	CIE: 100	
04	ESC	<tbd>	Problem Solving using Procedural Programming	3	0	2	1	4	30	10	60	CIE: 100	
05	PCC	<tbd>	Discrete Structures	2	0	0	1	2	30	20	50	--	--
06	VSE C	<tbd>	Web Design	1	0	2	1	2	CIE: 100			CIE: 100	
07	IKS	<tbd>	Communication Skills	2	0	0	0	2	CIE: 100			--	--
08	CCA	<tbd>	Liberal Learning -2	0	0	2	0	1	--	--	--	CIE: 100	
Total				15	00	12	06	21					

Legends: **L**-Lecture, **T**-Tutorial, **P**-Practical, **S**-Self Study, **Cr**-Credits
ISE-In-Semester-Evaluation, **ESE**-End-Semester-Evaluation, **MSE**-Mid-Semester Evaluation, **TA**-Teachers' Assessment, **CIE**-Continuous-Internal-Evaluation

Exit option to qualify for Certification: 2 Courses on Web Technologies

S.Y. B. Tech.
Computer Science and Engineering
[Level 5, UG Diploma] Semester -III

Sr. No.	Course Type	Course Code	Course Name	L	T	P	S	Cr	Evaluation Scheme (Weightages in %)				
									Theory			Laboratory	
									MSE	TA	ESE	ISE	ESE
01	PCC	<tbd>	Microprocessors	3	0	2	1	4	30	10	60	CIE: 100	
02	PCC	<tbd>	Principles of Programming Languages	3	0	0	1	3	30	10	60	--	--
03	PCC	<tbd>	Data Structures and Algorithms	3	0	2	1	4	30	10	60	CIE: 100	
04	OE	<tbd>	Open Elective-1	2	0	0	1	2	30	20	50	--	--
05	AEC	<tbd>	Indian language	2	0	0	0	2	CIE: 100			--	--
06	VEC	<tbd>	Environment Studies	2	0	0	0	2	CIE: 100			--	--
07	CEA	<tbd>	Community Engagement Activity (CEA)/Field Project	--	--	--	0	2	--	--		CIE: 100	
08	HS MC	<tbd>	Entrepreneurship	2	0	0	0	2	CIE: 100			--	--
Total				17	00	04	04	21					

[Level 5, UG Diploma] Semester -IV

Sr. No.	Course Type	Course Code	Course Name	L	T	P	S	Cr	Evaluation Scheme (Weightages in %)				
									Theory			Laboratory	
									MSE	TA	ESE	ISE	ESE
01	PCC	<tbd>	Object Oriented Programming & Design	3	0	2	1	4	30	10	60	CIE: 100	
02	PCC	<tbd>	Computer Organization	3	0	0	1	3	30	10	60	--	--
03	PCC	<tbd>	Theory of Computation	2	1	0	1	3	30	10	60	--	--
04	VEC	<tbd>	Constitution of India	2	0	0	0	2	CIE: 100			--	--
05	HS MC	<tbd>	Economics	2	0	0	0	2	CIE: 100			--	--
06	OE	<tbd>	Open Elective-2	2	0	0	1	2	30	20	50	--	--
07	VSE C	<tbd>	Development Tools Laboratory	0	0	2	0	1	--	--		CIE: 100	
08	MD M	<tbd>	Multidisciplinary Minor-1	3	0	0	1	3	30	10	60	--	--
Total				18	00	04	04	20					

T.Y. B. Tech.
Computer Science and Engineering
[Level 4.5, UG Certificate] Semester -V

Sr. No.	Course Type	Course Code	Course Name	L	T	P	S	Cr	Evaluation Scheme (Weightages in %)				
									Theory			Laboratory	
									MSE	TA	ESE	ISE	ESE
01	PCC	<tbd>	Design and Analysis of Algorithms	3	0	2	1	4	30	10	60	CIE: 100	
02	PCC	<tbd>	Computer Networks	3	0	2	1	4	30	10	60	CIE: 100	
03	PCC	<tbd>	Operating Systems	3	0	2	1	4	30	10	60	CIE: 100	
04	PEC	<tbd>	DE-1	3	0	0	1	3	CIE: 100			--	--
05	OE	<tbd>	Open Elective-3	2	0	0	1	2	30	20	50	--	--
06	OJT	<tbd>	Internship	--	--	--	0	3	--	--	--	CIE: 100	
07	MDM	<tbd>	Multidisciplinary Minor-2	4	0	0	1	4	30	10	60	--	--
Total				18	00	06	06	24					

[Level 4.5, UG Certificate] Semester -VI

Sr. No.	Course Type	Course Code	Course Name	L	T	P	S	Cr	Evaluation Scheme (Weightages in %)				
									Theory			Laboratory	
									MSE	TA	ESE	ISE	ESE
01	PCC	<tbd>	Database Management Systems	3	0	2	1	4	30	10	60	CIE: 100	
02	PCC	<tbd>	Artificial Intelligence	3	0	2	1	4	30	10	60	CIE: 100	
03	PCC	<tbd>	Cryptography and Network Security	3	0	0	1	3	30	10	60	--	--
04	PCC	<tbd>	DE-2	3	0	2	1	4	30	10	60	CIE: 100	
05	VSEC	<tbd>	Full stack development / Devops / Automation Testing	0	0	4	0	2	--	--	--	CIE: 100	
06	MDM	<tbd>	Multidisciplinary Minor-3	4	0	0	1	4	30	10	60	--	--
Total				16	00	10	05	21					

Exit option to qualify for B.Voc.: Mini Project / Internship

Final Year B. Tech.
Computer Science and Engineering
[Level 4.5, UG Certificate] Semester -VII

Sr. No.	Course Type	Course Code	Course Name	L	T	P	S	Cr	Evaluation Scheme (Weightages in %)				
									Theory			Laboratory	
									MSE	TA	ESE	ISE	ESE
01	PCC	<tbd>	Compiler Construction	3	0	2	1	4	30	10	60	CIE: 100	
02	PCC	<tbd>	Software Engineering	3	0	2	1	4	30	10	60	CIE: 100	
03	PEC	<tbd>	DE-3	3	0	0	1	3	30	10	60	--	--
04	PEC	<tbd>	DE-4	3	0	0	1	3	30	10	60	--	--
05	RM	<tbd>	Research Methodology	2	0	0	0	2	CIE: 100			--	--
06	OJT	<tbd>	Internship	-	-	-	-	3	--	--	--	CIE: 100	
07	MD M	<tbd>	Multidisciplinary Minor-4	3	0	0	1	3	30	10	60	--	--
Total				18	00	04	05	22					

[Level 4.5, UG Certificate] Semester -VIII

Sr. No.	Course Type	Course Code	Course Name	L	T	P	S	Cr	Evaluation Scheme (Weightages in %)				
									Theory			Laboratory	
									MSE	TA	ESE	ISE	ESE
01	PEC	<tbd>	DE-5	3	0	0	1	3	30	10	60	--	--
02	PEC	<tbd>	DE-6	3	0	0	1	3	30	10	60	--	--
03	OJT /VS EC	<tbd>	Internship/ Project	-	-	-	-	6				CIE: 100	
Total				18	00	04	05	23					

MULTIDISCIPLINARY MINORS

Semester	Course Code	Course Title	L	T	P	Cr
IV	MD M-01	Data Structures, Files and Algorithms	3	0	0	3
V	MD M-02	Object Oriented Programming	4	0	0	4
VI	MD M-03	Fundamentals of Database Management Systems	4	0	0	4
VII	MD M-04	Fundamentals of Data Science	3	0	0	3
Total						14

OPEN ELECTIVES LIST
(offered to other departments)

Semester	Course Code	Course Title	L	T	P	Cr
III	OE-01	Data Analytics	2	0	0	2
IV	OE-02	Fundamentals of Operating Systems/ Fundamentals of Algorithms	2	0	0	2
V	OE-03	Fundamentals of Machine Learning	2	0	0	2
Total						6

DEPARTMENT ELECTIVES

	DE-1	DE-2	DE-3	DE-4	DE-5	DE-6
Tracks	Sem-5	Sem-6	Sem-7	Sem-7	Sem-8	Sem-8
System Software/ Hardware	Cloud Computing PCAP: Parallel Computer Architecture and Programming	POSIX programming	SA: System Administration GPU Computing	MT: Multicore Technology ADBMS: Advanced Database Management Systems	ES: Embedded Systems DS: Distributed Systems	S&V: Storage and Virtualization
Networking and Security	MAN: Mobile and Ad-hoc Networks Remote Sensing	BCT: BlockChain Technologies GIS	IOT: Internet of Things	DS: Distributed Systems	CFLP: Computer Forensics and Data Recovery	CS: Cyber Security
Artificial Intelligence	Data-Science	ML: Machine Learning	Data Visualization Techniques	NLP: Natural Language Processing	DL: Deep Learning	Generative AI
Algorithms and Programming	ADS: Advanced Data Structures	FP: Functional Programming	OOMD: Object Oriented Modeling and Design	Parallel Algorithms	G&M: Graphics and Multimedia	SDP: Software Design Patterns

(PCC-02) Microprocessors

Teaching Scheme

Lectures: 3 Hrs/ Week

Labs: 2Hr. / Week

Self Study: 1 Hour/week

Evaluation Scheme

Theory: MES:30 Marks, TA:10 Marks ESE:60 Marks

Laboratory: CIE:100 Marks

Course Outcomes

Upon completion of this course students will be able to:

1. **Design** a microcomputer around 8086b CPU in minimum mode
2. **Explain** importance of power on reset circuit in the microcomputer
3. **Develop** assembly language programs for 8086 CPU
4. **Explain and write** programs for dedicated interrupts (Break Point, Overflow) of 8086
5. **Design and Test** interfacing of 8255, 8279, 8259, 8251,DMA and interfacing of 8237 to 8086
6. **Design** maximum mode CPU module for coprocessor configuration and multiprocessor system for local bus and system bus

Course Contents

Design of Microcomputer: Von Pneumann's Principle, 8086 Architecture, Advantages of pipelining and segmentation, Simultaneityand concurrency, Demultiplexing of address and data bus, ALE, Even and Odd Bank, BHE, Static memory organisation, Design of memory map, Design of Interfacing of static memory to 8086.Reset in, Power on Reset Circuit, Bus cycle, Instruction cycle, Organising program for Basic Input Output System or Boot Strap loader after reset, clock input, 8284 clock driver, Design of minimum mode CPU module.

[8 Hrs]

Programming with 8086: Programming modelof 8086,Physical Address, Segment Address and Logical address/offset, Addressing modes, Instruction codes, default segment assignment, Instruction Groups, Flags, Important instructions of the group, flags, prefixes. Programming in Assembly Language of 8086. I/O interfacing. I/O mapped I/O versus Memory mapped I/O. In and OUT. Interfacing of 8 bit input port and outport in I/O mapped I/O mode. Disadvantage of memory mapped I/O in design of multitasking Microcomputer.

[8 Hrs]

Interrupts: Interrupt structure of 8086, Dedicated Interrupts, flags associated with interrupts, hardware interrupts, non vectoredinterrupt, INTR and INTA, Software Interrupts

[4 Hrs]

I/O Interfaing: Program driven data transfer versus Interrupt driven data transfer, Interfacing design and demonstrating modes of 8255, simple I/o, Strobed input and output, bidirectional strobed I/O, BSR, 8279, encoded and decoded scan, 2 key lockout and N key rollover, sensor matrix, strobed keyboard, 8259, EOI, Non specific and specific EOI, Priority structure, Initialisation and Operational Command words, cascaded mode, buffered and non buffered

[6 Hrs]

Serial Communication, DMA mode: Serial I/O, Asynchronous and synchronous mode, design of serial communication using 8251 USART and RS232C drivers, Direct Memory Access and Interfacing of 8237 **[4 Hrs]**

Multiprocessing in 8086: Design of maximum mode CPU module, 8288 bus controller and 8289 bus arbiter, coprocessor configuration, Introduction to NDP8087, Resident Bus and System bus, loosely coupled and closely coupled configuration. Protected mode of 80286 for improved memory management and task switching for multiprogramming or multitasking, PVAM, Register set, segment descriptors, selector, virtual address space, single level and multiple level tasks, cache memory and its management **[8 Hrs]**

Self Study Unit:

- *Emu86 or suitable assembler, Study of assembler directives, Instruction templates, No. of Bus cycles and clock cycles for execution of instructions, All instructions of 8086, Prefixes, Programming practice in assembly language of 8086,*
- *Function calls of DOS and BIOS Interrupts for display and keyboard of IBM PC*
- *Interfacing and programming for 4x4 keymatrix and 4 seven segment multiplexed displays to 8255*

[8 Hrs]

Note:

- All the Course outcomes 1 to 3 will be judged by 75% of the questions and outcomes 4, 5 and 6 will be judged by 25 % of questions.
- To measure CO3 and CO4, some questions may be based on self-study topics

Text Book:

John P Uffenbeck, The 8086/8088 Family Design, Programming and Interfacing, PHI

Reference Books

Yu-Cheng Liu, Glenn A. Gibson, Microcomputer Systems: The 8086/8088 family Architecture, Programming and Design II Edition

Principles of Programming Languages

Teaching Scheme

Lectures: 2Hrs/ Week

Evaluation Scheme

Theory: MES:30 Marks, TA:10 Marks ESE:60 Marks

Course Outcomes

Students will be able to:

1. Explain, compare and discuss different language translation mechanisms.
2. Explain fundamental concepts of different programming paradigms.
3. Discuss and analyze factors that impact implementation of different programming language concepts and tradeoffs involved.
4. Demonstrate ability to write simple programs using exception handling and event driven programming concepts.
5. Suggest a suitable programming paradigm for a given problem.
6. Implement the solution for a given problem using different programming paradigms.

Course Contents

Preliminaries: Reasons for Studying Concepts of Programming Languages. Programming Domains. Language Evaluation Criteria. Influences on Language Design: computer architecture. Language Categories. Language Design Trade-Offs

[5 Hrs]

Implementation Methods :Compilation, Interpretation, Hybrid Implementation, Preprocessors. Programming Environments. Evolution of the Major Programming Languages (Pseudocodes, Assembly, C, C++, FORTRAN, Java). Introduction to assembly language instructions.

[5 Hrs]

Syntax and Semantics Lexical and Syntax Analysis. Names, Bindings, and Scopes, Type checking, Strong Typing. Type Conversions. Short-Circuit Evaluation

[8 Hrs]

Statement-Level Control Structures Subprograms: Introduction. Fundamentals of Subprograms. Design Issues for Subprograms. Local Referencing Environments. Parameter-Passing Methods

[6 Hrs]

Exception Handling: Basic Concepts. Design Issues. Exception Handlers . Binding Exceptions to Handlers. **Event Handling:**Basic Concepts. Event handling with Programming Language. Comparison of Exception handling & Event Handling. GUI development

[6 Hrs]

Programming Paradigms:

Introduction to Logic Programming. Introduction. Applications of Logic Programming. Introduction to Functional Programming. Introduction. Introduction to LISP. Garbage Collection Algorithms. A Comparison of Functional and Imperative Languages. Introduction to Procedural Programming. Introduction to Object Oriented Programming

[10Hrs]

Self Study topics:

Python as a multi-paradigm language for functional and logic programming. Case study of Ada, PERL, Go and Ruby for their features. Comparison of efficiency of compiled and interpreted languages in the form of assignments.

[12 Hrs]

Suggested List of Assignments:

1. Compile a C program with *gcc* with various options like -S, --save-temps. List the various temporary files and the meaning of their contents.
2. Use *objdump* to read the object file of a C program and demonstrate how a stack frame is built and destroyed.
3. Use *gdb* to read assembly code for C program to find factorial of a number with recursion and demonstrate how parameters are passed using pass by value.
4. Explain how pass by reference works, using assembly code generated using *g++ -S* on a C++ program.
5. Write a program to raise an exception and handle it.
6. Write a function in LISP to find *n*th element from a list of *m* elements.
7. Implement a decision support system for course tracking at a institute using PROLOG. A course is floated by a teacher from a particular department. Various students will enroll a course. Single student can enroll for multiple courses (maximum five per course) provided the student clears the prerequisite courses. A teacher can teach at the most two courses in a semester.
8. Demonstrate how a program, for example on to find the *n*'th element of a list, can be written differently using procedural, object-oriented, logic and functional programming approaches.

Text Books

- Scbesta R., "Concepts Of Programming Languages", 4th Edition, Pearson Education, ISBN-81-7808-161-X
- T. W. Pratt , "Programming Languages", 4th Edition ,Prentice-Hall Of India, ISBN 9780130287199.

Reference Books

- Ghezzi C, Milano P, Jazayeri M., "Programming Languages Concepts", 3rd Edition, John Wiley and Sons Pvt. Ltd (WSE), ISBN - 0195113063
- Michael L. Scott "Programming Language Pragmatics", ELSEVIER Publication, ISBN: 81-8147-370-1
- Roosta S., "Foundations of Programming Languages", Thomson, Brooke/Cole, ISBN 981-243-141-1
- Sethi R., "Programming Languages concepts & constructs", 2nd Edition, Pearson Education, ISBN 81 - 7808 - 104 - 0

(PCC-04) Data Structures and Algorithms

Teaching Scheme

Lectures: 3Hrs/ Week
Labs: 2 Hr. / Week

Evaluation Scheme

Theory: MES:30 Marks, TA:10 Marks ESE:60 Marks

Laboratory: CIE:100 Marks

Course Outcomes

Students will be able to:

1. Discuss, illustrate, and analyze properties of various data structures.
2. Write neat code by selecting appropriate data structure and demonstrate a working solution for a given problem.
3. Analyze clearly different possible solutions to a program and select the most efficient one.
4. Discuss, compare and implement algorithms for various operations in different implementations of list, stack, queue, tree, graph, matrices, heap, etc.
5. Write an application requiring an effort of at least 1000 lines of code to demonstrate a good working solution.
6. Demonstrate the ability to write reusable code and abstract data types.

Course Contents

Introduction: Data. Data types. Object, data structure and abstract data types (ADT). Characteristics of an algorithm. Analyzing programs. Frequency count. Time and space complexity. Big 'O' and Ω notation. Best. average and worst cases. Dangling pointers and garbage memory.

[4 Hrs]

Arrays, Files and Searching: Searching: linear and binary search algorithm, and their analysis. Hashing: hashing functions, chaining, overflow handling with and without chaining, open addressing: linear. quadratic probing. Files handling: text and binary files, use of various libraries for handling files.

[4 Hrs]

Stacks and Queues: Stack and queue as ADT. Operations on stack and queue. Implementations using arrays and dynamic memory allocation. Application of stack for expression evaluation, expression conversion. Recursion and stacks. Problems like maze and knight's tours

[8 Hrs]

Lists: List as ADT. Concept of linked organization of data against linked list. Singly linked list, doubly linked list, circular linked list. Representation & manipulations of polynomials/sets using linked lists. Dynamic memory management. Representation of sparse matrix. Addition and transpose of sparse matrix.

[6 Hrs]

Priority queues and Heap: Priority Queues. Max and Min Heap. Operations on heap, Heapsort.

[3 Hrs]

Sparse Matrices: Matrix operations like addition and multiplication. Sparse matrices concept. Different implementations of sparse matrices. Operations like addition and multiplication. Time complexity comparisons.

[3 Hrs]

Trees and Graphs: Basic terminology. Binary trees and its representation. Binary tree traversals (recursive and non recursive) and various operations. Insertion and deletion of nodes in binary search tree. Representation of graphs using adjacency matrix, adjacency list. Implementation of algorithms for traversals implementing Kruskal's. Prim's algorithms. Single source shortest paths using Dijkstra's algorithm. Applications of graphs and trees.

[8 Hrs]

Time Complexity Analysis, Algorithm Design: Verification of programs, invariants, assertions, proof of termination. Best, Average and Worst case analysis of various algorithms; Designing data structures for specific applications.

[4 Hrs]

Self Study topics

Applications like unlimited precision numbers and operations on them, 8-Queens Problem, Air Traffic controller, Solving maze. Applications of trees. AVL Trees: Introduction, Properties, Balance Factor, Operations like insert, rotate and delete, Red Black Trees: Properties, Operations like insert, delete and rotate.

[12 Hrs]

Suggested List of Assignments:

1. Write a program to remove duplicate doubles from an array of doubles. In the program, write a function which accepts an array of doubles and removes the duplicates from the array and has return type void.
2. Compare the time complexity of two sorting algorithms, by following the given steps. Create a set of data files with count of integers varying into thousands and millions. Sort the files using both the algorithms. Plot graph of the time taken by both the programs using tool like gnuplot. Compare the graphs and comment on the time complexity theoretically predicted and practically observed.
3. Write a function which evaluates an infix expression, without converting it to postfix. The input string can have spaces, (,) and precedence of operators should be handled.
4. Implement a queue (that is write queue.c and queue.h only) of characters, such that on an enqueue, the char is added at the end of queue, and on a dequeue the first element is taken out, but the queue uses only a 'head' pointer and not a 'tail' pointer.
5. Write an data type called "Integer". The data type should represent integers of unlimited length.
6. Write a sorting program with the following features: Reads data from a text file and sorts it alphabetically by default. If the file has data in rows and columns (separated by space or tab) then allows sorting on a particular column. Allows any sort using numeric or alphabetical ordering.
7. Write the following functions for a binary search tree implementation: Searches the maximum value in the tree, preorder traversal without using recursion, Search the string in the tree and returns a pointer to the node, print the binary tree so that it looks like a tree.
8. Write code to list leaf nodes, non-leaf nodes and level of all nodes in a given binary tree.
9. Write a code for level order traversal of a binary tree with and without stack.
10. Start with an empty AVL tree and perform series of insertions like : December, January, April, March, July, August, October, February, November, May, June. Display the tree.

11. Implement a sparse matrix with operations like initialize empty sparse matrix, insert an element, sort a sparse matrix on row-column, add two matrices and return the result as a matrix, transpose a matrix, etc.
12. Develop C functions to insert and delete into/from a max heap under the assumption that a dynamically allocated array is used, the initial capacity of this array is 1, and array doubling is done whenever we are to insert into a max heap that is full.
13. Implement Heap sort algorithm on a set of records, with a specified key.
14. Write a graph implementation, using adjacency lists and demonstrate Dijkstra's algorithm on it.
15. Write a program to read a map stored in a text file with (city1, city2, distance) comma separated values. Build a graph using this data. Print all pairs shortest paths information between all pairs of cities.
16. Implement DFS and BFS on a Graph.
17. Write a program to find all connected components of a Graph, on a map specified in a text file as Source, Destination, distance comma separated values.
18. Develop a hash table implementation in which overflows are resolved using chaining. Read a set of records from a file, insert them into hash table, then perform a set of searches using use supplied data and show the search results.
19. Implement a dictionary using a sparse matrix data structure.

eMini-project: Write an application of your own demonstration your skills in defining a problem, writing down the requirements carefully, designing a modular solution with clear separation of abstract data types and their use, design of proper function prototypes and division of work among functions. The application can be a unix command re-implemented (e.g. cut, find, tar, fdupes, bc, etc.), reimplementation of C library functions, memory allocator, a simple game using libraries like n-curses or SDL, games like sudoku or chess, or an application to manage institutions like hospitals, colleges, shops, etc.

Text Books

- E. Horowitz, S. Sahni, S. Anderson-freed, "Fundamentals of Data Structures in C", Second Edition, University Press, ISBN 978-81-7371-605-8
- B. Kernighan, D. Ritchie, "The C Programming Language", Prentice Hall of India, Second Edition, ISBN 81-203-0596-5
- Y. Langsam, M. Augenstein and A. Tannenbaum, "Data Structures using C", Pearson Education Asia, First Edition, 2002, ISBN 978-81-317-0229-1

Reference Books

- Ellis Horowitz, S. Sahni, D. Mehta "Fundamentals of Data Structures in C++", Galgotia Book Source, New Delhi 1995 ISBN 16782928
- Jean-Paul Tremblay, Paul. G. Soresan, "An introduction to data structures with Applications", Tata Mc-Graw Hill International Editions, 2nd edition 1984, ISBN-0-07-462471-7

(OE-01) Data Analytics

Teaching Scheme

Lectures: 2Hrs/ Week

Evaluation Scheme

Theory : CIE:100 Marks

Course Outcomes

Students will be able to:

1. **Know** basics of Data Analytics, people associated with it, roles and responsibilities to be an data Analyst.
2. **Learn** and **apply** how to interpret data in Python using multi-dimensional arrays in NumPy.
3. **Learn** and **apply** to manipulate DataFrames in Pandas.
4. **Apply** visualization libraries in Python like Matplotlib and Seaborn to gain insight of dataset.
5. **Learn** latest data analytics tools excel, Plotly, Power BI, Tableau.

Course Contents

Introduction to Data Analytics: Introduction to Data Analytics, Need of data Analytics, types of data analytics, Challenges, different types of data, data sources, data collection, data integration, data wrangling, role of Data Engineers, Data Analysts, Data Scientists, Business Analysts, and Business Intelligence Analysts, roles, responsibilities and skill sets required to be a Data Analyst.

[6 Hrs]

Introduction to Python Libraries-NumPy: Installation, Basic operation: Arithmetic, Matrix Product, Increment, Decrement, Aggregate function, Indexing, Slicing and Iterating an array, Conditions and Boolean Arrays, Shape manipulations, Array manipulation – multidimensional array

[6 Hrs]

Pandas Library: Installation of Pandas, Testing pandas installation, Introduction to pandas data structure- the series, data frame, Index objects, other functionalities on Indexes-Reindexing, Dropping, Operations on DataFrame and Series- merging, combining, pivoting, removing, data transformation, Data aggregation, Reading and writing data to CSV file, random sampling, group iteration.

[7 Hrs]

Data Visualization with Matplotlib and Seaborn libraries:Pyplot, plotting window, adding elements to chart- text, grid, legend, saving charts, Handling data values, Line chart, Histogram, Bar chart, pie chart, scatter plot, box plot, heat map.

[7 Hrs]

[Self Study]

Tools for Data Analytics:Introduction to data analytics tools, collection of maps, diagrams, charts designed to gather, interpret, and visualize data across diverse applications, how to Choose the right data analysis tool, data visualization with different free open source tools.

[12 Hrs]

Text Books

- Fabio Nelli, "Python Data Analytics with Pandas, Numpy and Matplotlib", ISBN: 978-1-4842-3913-1, DOI: 10.1007/978-1-4842-3913-1, Publisher: Apress, Year: 2018
- Bharti Motwani, "Data Analytics using Python ", Publisher: Wiley, ISBN: 8126502959

Reference Books

- Vincent Granville, "Developing Analytic Talent -Becoming a Data Scientist", Publisher: Wiley, 2014, ISBN: 9781118810095.
- Glenn J. Myatt, Wayne Johnson, "Making Sense of Data I", Second Edition, Publisher: Wiley, ISBN 978-1-118-40741-7
- Glenn J. Myatt, Wayne Johnson, "Making Sense of Data II", Publisher: Wiley, ISBN 978-0-470-22280-5

(PCC-05) Object Oriented Programming & Design

Teaching Scheme:

Lectures: 3 Hr/week
Practical: 2Hr/week

Examination Scheme:

Theory: MES:30 Marks, TA:10 Marks
ESE:60 Marks
Laboratory: CIE:100 Marks

Course Outcomes

Students will be able to:

1. Design a class hierarchy using object oriented thinking for a given problem.
2. Create object oriented application code for a given problem.
3. Write small pieces of code demonstrating various object oriented programming concepts.
4. Compare, annotate, and comment on various object oriented programming concepts.
5. Demonstrate ability to write concurrent programs for given problems.
6. Apply different UML diagrams for given problems.

Course Contents

Introduction: Various programming paradigms: Procedural, object-oriented, logic and functional, concurrent programming. Classes, Objects, Methods; Data types provided by OO languages; Abstract Data Types;

[8 Hrs]

Abstraction Mechanisms: Encapsulation; Constructors, Destructors; Polymorphism; Access specification: Private, public, protected members;

[8 Hrs]

Inheritance: Types of inheritance: single, multilevel, multiple, hierarchical and hybrid; Class hierarchies; Virtual Functions; Virtual Base class;

[6 Hrs]

Standard Libraries: Templates; Generic Programming using generic function and class; Packages; Interfaces. Iterators; Containers.

[6 Hrs]

Exception Handling & Multi-Threaded Programming: Exception handling, Exception types, Concurrent Programming, Basic Concepts of Concurrent Programming, Threads.

[6 Hrs]

Introduction to OOAD : Unified Process – UML diagram, use case, class diagrams, interaction diagrams, state diagrams – activity diagrams – package, component and deployment diagrams. Design: Unified Modelling language; use case diagrams; Class Diagrams;

[6 Hrs]

(Self Study Unit)

Design Patterns: Introduction to Design Patterns, Types of Design Patterns, Application on various design patterns.

[12 Hrs]

Suggested List of Assignments

This is only a suggestive list. Instructor is encouraged to update the list of assignments and projects. The purpose of the assignments should be to lead to a meaningful project. The students should be encouraged to build different projects.

1. Define a class to represent a bank account. Include the following details like name of the depositor, account number, type of account, balance amount in the account. Write methods to assign initial values, to deposit an amount, withdraw an amount after checking the balance, to display name, account number, account type and balance.
2. Write a program to implement following. Create a base class called person consisting of name and code. Create 2 child classes a. Account with member pay and b. Admin with experience and inherit the base class. Create a class Employee with name, code, experience and pay by inheriting the above class.
 - a. Write Python script to display
 - b. Current date and time,
 - c. Current year,
 - d. Month of year,
 - e. Week number of the year,
 - f. Weekend of the week,
 - g. Day of year,
 - h. Day of the month and Day of week.
3. Write a program that has an abstract class Polygon. Derive two classes Rectangle and Triangle from Polygon and write methods to get details of their dimensions hence calculate the area.
4. Write a menu driven to read, add, subtract, multiply, divide and transpose two matrices.
5. Write a program that has a class TIME. Enter the time when a user started an online test and completed the test. Subtract the two-time values and display the duration in which the test was completed. Throw exceptions whenever need arises (like invalid data, or if start time is greater than completion time).
6. Construct a class diagram for a geometrical document. Add at least 10 relationships (associations and generalizations). Use associations and association end names wherever required. Also use qualified associations and show multiplicity. You do not need to show attributes or operations. As you prepare the diagrams, you may add classes. Be sure to explain your diagrams.
7. Scenario: An extension ladder has a rope, pulley, and latch for raising, lowering, and locking the ex-tension. When the latch is locked, the extension is mechanically supported and you may

- safely climb the ladder. To release the latch, you raise the extension slightly with the rope. You may then freely raise or lower the extension. The latch produces a clacking sound as it passes over rungs of the ladder. The latch may be reengaged while raising the extension by reversing direction just as the latch is passing a rung. Construct a state diagram of an extension ladder.
8. Draw a Scenario: A hockey league is made up of at least four hockey teams. Each hockey team is composed of six to twelve players, and one player captains the team. A team has a name and a record. Players have a number and a position. Hockey teams play games against each other. Each game has a score and a location. Teams are sometimes lead by a coach. A coach has a level of accreditation and a number of years of experience, and can coach multiple teams. Coaches and players are people, and people have names and addresses. Construct a UML Class Diagram representing the above problem domain for a hockey league.

Text Books

- Cay S Horstmann and Gary Cornell, Core Java Vol-1 and Vol-2, 9th Edition, Pearson Education India, ISBN-10: 9332518904 and 9332518890
- Kenneth Barclay and John Savage, Object-Oriented Design with UML and Java, Elsevier Science. ISBN: 9780080497556
- Ronald Leach, Object-Oriented Design and Programming with C++: Your Hands-On Guide to C++ , Academic Press, ISBN: 9781483214122
- M.Ben Ari, Principles of Concurrent Programming, Prentice-Hall International, ISBN:0137010788

Reference Books

- Herbert Schilt, "JAVA Complete Reference", 7th Edition, Tata McGraw Hill, ISBN: 9780070636774
- Scott W. Ambler ,The Elements of UML (TM) 2.0 Style, ISBN- 978-0521616782
- Sharon Zakhour, Scott Hommel, Jacob Royal, Isaac Rabinovitch, Tom Risser, Mark Hoeber, "The Java Tutorial, "Addison Wesley Professional, 2006, Print ISBN-10: 0-321-33420-5
- Eckel B., "Thinking in Java", 3rd Edition, Pearson Education, 2012
- E. Balagurusamy, Object Oriented Programming with C++, 6th Edition, McGraw Hill, ISBN-10: 125902993X

(PCC-06) Computer Organization

Teaching Scheme

Lectures: 2Hrs/ Week

Evaluation Scheme

Theory: MES:30 Marks, TA:10 Marks ESE:60 Marks

Course Outcomes

Students will be able to:

1. **Analyze** Performance aspects, Instruction set architecture, and Functional Units.
2. **Apply** different computer arithmetic principles for implementation of functional units.
3. **Interpret** the working of memory hierarchy of computer system.
4. **Justify** the need of paging in virtual memory and secondary storage with its advantages.
5. **Measure and analyze** features of multiprocessor systems like bus arbitration, Instruction pipelining, and RISC & CISC.
6. **Examine Input/Output** including interfaces, buses, interrupt handling mechanisms, and I/O controllers

Course Contents

CPU Architecture: Performance understanding, Amdahl's Law, Benchmarking, Flynn's Classification, Instruction format, control signals in CPU, micro program control unit and hardwired control unit, ALU & sequencer, look ahead carry generator, MIPS ISA

[6Hrs]

Arithmetic: Integer Arithmetic-multiplication, Booth's Algorithm, division algorithm; Floating point number representation, and floating-point arithmetic

[6Hrs]

Memory: Dynamic RAM organization, Cache memory & basic cache optimizations, cache coherence & MESI protocol, virtual memory, secondary storage, MBR and GPT hard disks, RAID, File system FAT

[8Hrs]

System and memory map: Closely coupled and loosely coupled multiprocessor systems, bus arbitration, co-processor, lower 1MB memory map

[7Hrs]

Instruction Pipelining: Basic concepts and issues, Introduction to the basic features & architecture of RISC & CISC processors, super scalar processor, MIPS pipeline

[7Hrs]

Multiprocessing: Symmetric multiprocessing (SMP), Asymmetric multiprocessing (AMP), Hardwired based multithreading approaches and examples, Different examples of computer organization as per Flynn's Classification

[5Hrs]

Self Study

Closely coupled and loosely coupled multiprocessor systems, Differences in RISC & CISC approaches, Differences in FAT and NTFS, Programs exhibiting Locality of Reference, Dependence analysis and hazard detection using different resources.

[12Hrs]

Text Books

- William Stallings, Computer Organization and Architecture, 11/E ISBN- 9781292420103/ 9781292420080, Pearson Education, Global Edition, 2021-22.
- Carl Hamacher, Zvonko Vranesic, Safwat Zaky, and Naraig Manjikian, Computer Organisation, 6th Edition, ISBN- 978-0-07-338065-0/ 0-07-338065-2, McGrawHill, 2011-12.

Reference Books

- D. Patterson, J. Hennessy, Computer Organization and Design: The Hardware Software Interface, RISC V Edition, ISBN- 978-0-12-812275-4, Morgan Kauffman, 2018.
- Liu & Gibson, Microcomputer Systems, Second Edition, ISBN: 978-81-203-0409-3, PHI, 1985.
- John Uffenbeck, THE 8086/ 8088 FAMILY: Design, Programming, and Interfacing, EEE, ISBN- 0132467526/ 9780132467520, PHI, 1987.

(PCC-06) Theory of Computation

Teaching Scheme

Lectures: 2Hrs/ Week

Tutorial: 1 Hr / Week

Self-Study: 1 Hr / Week

Evaluation Scheme

Theory: MSE: 30 Marks, TA: 10 marks

ESE: 60 Marks

Course Outcomes

Students will be able to:

1. Design appropriate automata for modeling the solution for various computational problems.
2. Apply transformation between multiple representations of automata/machines.
3. Make use of the pumping lemma to show that a language is not regular/context-free.
4. Distinguish different formal computing languages and classify their respective types.
5. Describe the limitations of a computing machine in terms of language recognition.

Course Contents

Finite Automata: Computability, and Complexity, Strings and Languages: symbol, alphabet, string, formal languages, Formal definition of a finite automaton, Designing finite automata, Non-determinism: Formal definition, Equivalence of NFAs and DFAs, Minimization of DFA.

[10 Hrs]

Regular Expressions: Regular Expressions: Formal definition of a regular expression, Equivalence with finite automata, Closure properties of regular languages, the pumping lemma for regular languages.

[4 Hrs]

Context-Free Languages: Context-free Grammars: Formal definition, Designing context-free grammars, Parse Trees, Ambiguity, Chomsky Normal Form. Pushdown Automata: Formal definition, Examples.

[6 Hrs]

Turing Machines and Undecidability: Turing Machines Formal definition, Examples, Decidability and Undecidability: Decidable Languages: Decidable problems concerning regular languages, Decidable problems concerning context-free languages, Undecidable problems, The Halting Problem.

[6 Hrs]

Topics for Self study: Basic Mathematical notations and terminologies for set theory. Closure Properties of context-free languages, the pumping lemma for context-free languages. Variants of Turing Machine: Multi-tape Turing machines, Nondeterministic Turing machines, Enumerators.

[6 Hrs]

Text Books:

- Michael Sipser, "Introduction to the Theory of Computation", 3rd Edition, 2013, Cengage Learning Publications, ISBN-13: 978-1133187790.
- John. E. Hopcroft, Rajeev Motwani, J. D. Ullman, "Introduction to Automata Theory, Languages, and Computations", 3rd Edition, 2009, Pearson Education Publisher, ISBN-10: 0321455363 / ISBN-13: 978-0321455369

Reference Books:

- E. V. Krishnamurthy, "Theory of computer science", 2004, Affiliated East Press Publications, ISBN-10: 038791255X / ISBN-13: 978-0387912554.
- Dexter C. Kozen, "Automata and Computability", 1997, Springer Verlag Publications, ISBN 0-387-94907-0.
- Harry Lewis, Christos H. Papadimitriou, "Elements of the Theory of Computation", 2nd Edition, 1997, Prentice-Hall Publications, ISSN 0891-4516.
- John Martin, "Introduction to Languages and Theory of Computations", 4th edition, 2010, McGraw-Hill Publications, ISBN 978-0-07-319146-1 / MHID 0-07-319146-9.

(OE-02) Fundamentals of Operating Systems

Teaching Scheme:

Lectures: 2 Hrs / Week

Self Study: 1 Hrs / Week

Evaluation Scheme:

MSE: 30 marks

TA: 20 marks

ESE: 50 marks

Course Outcomes:

Upon completion of this course, participants will be able to:

1. Understand the structure of OS and basic architectural components involved in OS design.
2. Describe the mechanisms of OS to handle processes, threads, and their communication.
3. Analyze the memory, file, and resource management techniques.
4. Illustrate the role of paging, segmentation, and virtual memory in operating systems.

Course Contents

Introduction to Operating System (OS): Computer System, OS Operations, Virtualization, Distributed Systems, Kernel Data Structures, Computing Environments, OS Services, User and OS Interface, System Calls, System Services, OS Design, OS Structure. **[6 Hrs]**

Process Management: Scheduling, Operations, Interprocess Communication, Models of IPC, Thread, Multicore Programming, Multithreading Models, Implicit Threading, CPU scheduling, Scheduling Algorithms, Thread Scheduling, Multi-Processor Scheduling. **[6 Hrs]**

Process Synchronization: Critical-section, Peterson's Solution, Hardware Support, Mutex Locks, Semaphores, Classic Problems of Synchronization, Synchronization within the Kernel, Deadlocks: necessary conditions, Prevention, Avoidance, and Detection. **[6 Hrs]**

Memory Management: Logical and Physical Memory, Contiguous Memory Allocation, Paging, Swapping, Virtual Memory, Demand Paging, Page Replacement, Allocation of Frames, Thrashing, Secondary Storage: HDD Scheduling, Swap-Space Management. **[6 Hrs]**

File Management: File Concept, Access Methods, Directory Structure, Protection, Filesystem Structure, Directory Implementation, Allocation Methods, Free-Space Management, Recovery, Filesystem Mounting, Partitions and Mounting, File Sharing. **[6 Hrs]**

Self Study Unit: Case Studies of Linux OS: Design Principles, Kernel Modules, Process Management, Scheduling, Memory Management, File Systems.

Textbooks

- "Operating System Concept" by Abraham Silberschatz, Peter Baer Galvin, Greg Gagne, 10th Edition, 2021, Wiley, ISBN-978-1-119-32091-3.
- "Modern Operating System", by Andrew S. Tanenbaum, 5th Edition, 2022, Pearson Education, ISBN 13: 978-0137618873.

Teaching Scheme

Lectures: 2 Hrs/ Week

Evaluation Scheme

Theory:

Test1: 20 Marks, Test2: 20 Marks,

ESE: 60 Marks

Course Outcomes:

Upon completion of this course, participant will be able to:

1. Understand the structure of OS and basic architectural components involved in OS design.
2. Describe the mechanisms of OS to handle processes, threads, and their communication.
3. Analyze the memory, file, and resource management techniques.
4. Illustrate the role of paging, segmentation, and virtual memory in operating systems.

Course Contents

Introduction to Operating System (OS): Computer System, OS Operations, Resource Management, Security and Protection, Virtualization, Distributed Systems, Kernel Data Structures, Computing Environments, OS Services, User and OS Interface, System Calls, System Services, Linkers and Loaders, OS Design, OS Structure, Building and Booting an OS.

[8 Hrs]

Process Management: Concept, Scheduling, Operations, Interprocess Communication, models of IPC, thread, Multicore Programming, Multithreading Models, Implicit Threading, CPU scheduling, Scheduling Algorithms, Thread Scheduling, Multi-Processor Scheduling.

[8 Hrs]

Process Synchronization: Critical-section, Peterson's Solution, Hardware Support, Mutex Locks, Semaphores, Monitors, Classic Problems of Synchronization, Synchronization within the Kernel, POSIX Synchronization, Deadlocks: necessary conditions, Prevention, Avoidance, and Detection.

[8 Hrs]

Memory Management: logical and physical memory, Contiguous Memory Allocation, Paging, Swapping, Virtual Memory, Demand Paging, Page Replacement, Allocation of Frames, Thrashing, secondary storage: HDD Scheduling, Swap-Space Management, RAID Structure.

[8 Hrs]

File Management: File Concept, Access Methods, Directory Structure, Protection, File-System Structure, Directory Implementation, Allocation Methods, Free-Space Management, Recovery, Filesystem Mounting, Partitions and Mounting, File Sharing, Virtual File Systems, Remote File Systems.

[8 Hrs]

Self Study Unit

Case Studies and Analysis: Case studies of real-world operating systems or scenarios where operating system design and concepts have played a critical role. Students have to analyze these case studies, identifying the OS components involved, design decisions made, and their impact on system performance and reliability.

[8 Hrs]

Text Books

- "Operating System Concept" by Abraham Silberschatz, Peter Baer Galvin, Greg Gagne, 10th Edition, 2021, Wiley, ISBN-978-1-119-32091-3.
- "Modern Operating System", by Andrew S. Tanenbaum, 5th Edition, 2022, Pearson Education, ISBN 13: 978-0137618873.

(OE-02) Fundamentals of Algorithms

Teaching Scheme

Lectures: 2 Hrs/ Week

Evaluation Scheme

Theory:

CIE: 100 Marks

Course Outcomes

Students will be able to:

- Recall** and **comprehend** time and space complexity of algorithms and evaluate algorithmic efficiency in terms of runtime and memory usage using asymptotic analysis.
- Describe** the performance of recursive algorithms and analyze the correctness and efficiency of their solutions.
- Evaluate** and use appropriate algorithmic techniques, and design efficient solutions to various computational challenges.
- Apply** design technique that would be most suitable to solve a given problem, justify the technique used and analyze the resultant algorithm.

Course Contents

Basics of Algorithm: Introduction to algorithm, Classification of algorithms- Deterministic, non-deterministic, performance analysis of an algorithm, time and space complexity, asymptotic notations (O , Ω , θ notations), complexity analysis with examples.

[6 Hrs]

Design Techniques-I: Greedy Methods: Introduction, Knapsack problem, Job sequencing with deadlines, Dijkstra's Single source shortest paths, Minimum cost spanning tree: Prim's algorithm, Kruskal's algorithm, Optimal merge patterns.

[6 Hrs]

Design Techniques-II: Divide and Conquer: Introduction, Mergesort, Binary Search, Quicksort, Multiplication of two n -bit numbers.

[5 Hrs]

Design Techniques-III: Dynamic Programming: Introduction, All pairs shortest paths, Traveling salesperson problem, Longest common subsequence, Bellman-Ford algorithm.

[5 Hrs]

Graph Algorithms: Depth-first search (DFS), breadth-first search (BFS), Topological sort.

[4 Hrs]

Selected Algorithms: String Matching: The naïve string-matching algorithm, The Robin-Karp algorithm, The Knuth- Morris-Pratt algorithm;

[4 Hrs]

Self Study Unit:

Algorithmic Problem Solving Platforms:Activity: Solve algorithmic problems on platforms like LeetCode or HackerRank.

[8Hrs]

Text Books

9. Ellis Horowitz, Sartaj Sahni and Sanguthevar Rajasekaran, "Fundamentals of Computer Algorithms", Universities Press, 2nd edition (2008) , ISBN-13: 978-8173716126
10. Thomas Cormen, Charles Leiserson, Ronald Rivest and Clifford Stein, "Introduction to Algorithms", PHI, 3rd edition, ISBN-13: 978-8120340077

Reference Books

11. Gilles Brassard and Paul Bratley, "Fundamentals of Algorithmics", PHI, ISBN-13: 978 8120311312
12. Jon Kleinberg and Éva Tardos, "Algorithm Design", Pearson Education India, ISBN-13: 978-9332518643

(VSEC-01) Development Tools Laboratory

Teaching Scheme

Labs: 2 Hrs / Week

Suggested Evaluation Scheme

Laboratory: CIE 100 Marks

Course Outcomes

Students will be able to:

1. Develop an application in a group using GIT, demonstrating ability to work remotely, push, and pull.
2. Write a report in a specified format using LaTeX.
3. Demonstrate programming ability using Unix Shell.

Course Contents

LaTeX: Basic syntax, compiling and creating documents; Document structure, sections, paragraphs; packages, Math, Adding Images, Drawing images (using tools like Inkscape) Table of contents; Source code, graphs (using tools like Graphviz), Adding references, different templates, IEEE format, Bibliography

Shell Programming: Introduction to Linux commands, concept of shell, shell variables, `getcwd()` and `pwd`; Introduction to shell programming features: Variables declaration & scope, test, return value of a program, if-else and useful examples, for and while loop, switch case; Shell functions, pipe and redirection, wildcards, escape characters; Awk script: Environment and workflow, syntax, variables, operators, regular expressions, arrays, control flows, loops, functions, output redirections

GIT: Creating a project using git locally, add, commit, status, diff; branch and merge, GIT: cloning a remote repo, working with a remote repo – git push, pull, fetch; creating issues and pull requests; working on a project in a distributed fashion

Suggested List of Assignments:

- The lab incharge will impart instructions during a part of the lab session and assign the assignments based on it for an appropriate duration of days.
 1. Format a given essay using sections, paragraphs, headings in LaTeX.
 2. Format a given report in IEEE format using LaTeX.
 3. Write a shell program which reads a set of unspecified count of numbers and prints their sum and average.
 4. Write a shell program to extract a compressed file in any format (zip, tar.gz, tar.gz2, .tar, .bz2, .gz, .rar, .Z, .7z, etc)

5. Write a shell program to convert a CSV file of contacts, into a VCF file.
6. Write a shell program to sort all files stored in a given folder hierarchy, on their size.
7. Write a shell script to manage a todo list from command line. The script should be able to add, remove, list, sort, prepend, append, deduplicate todo-items
8. Write a program that scans a file line by line, splits each input line into fields later, compares input line/fields to pattern and performs action(s) on matched lines
9. Develop a program using git locally. E.g. add the exponent operator to the calculator program that you wrote. Demonstrate the ability to do git add, commit, status, diff.
10. Create a branch in your calculator program to do hexadecimal calculation. Write code and develop two branches. Merge the two to have a decimal/hex calculator. Demonstrate git branch, merge capability.
11. In a group of 3, create a github/gitlab repo. Raise issues, send pull requests, do the local and remote merges and finally get a synced local repo. For example, in the calculator project one student to become the developer, the other two to create issues and send pull requests for features like adding an operator, developing pulling those requests. Rotate the roles and repeat.

References:

LaTeX

- Leslie Lamport, "LaTeX: A document preparation system", User's guide and reference manual, 2nd Edition, 1994, by Addison-Wesley Professional. ISBN 0201529831, 9780201529838
- Stefan Kottwitz, "LaTeX Beginner's Guide: Create High-quality and Professional-looking Texts, Articles, and Books for Business and Science Using LaTeX, Packt Publishing, 2011. ISBN: 1847199860, 9781847199867
"<https://www.latex-project.org/>
- Introduction to LaTeX, MIT
<http://web.mit.edu/rsi/www/pdfs/new-latex.pdf>
- A simple guide to LaTeX - Step by Step
<https://www.latex-tutorial.com/tutorials/>

Shell

- Bash Guide for Beginners: <https://www.tldp.org/LDP/Bash-Beginners-Guide/Bash-Beginners-Guide.pdf>
- Bash Reference Manual <https://www.gnu.org/software/bash/manual/bash.pdf>
- Tutorials on Shell Programming <https://www.shellscript.sh/>
https://www.tutorialspoint.com/unix/shell_scripting.htm

GIT

Pro GIT Book <https://github.com/progit/progit2/releases/download/2.1.204/progit.pdf>

(MD M-02) Data Structures, Files and Algorithms

Teaching Scheme:

Lectures: 3 Hrs/week

Examination Scheme:

Theory: MES:30 Marks, TA:10, ESE: 60 Marks

Course Outcomes

Students will be able to:

- 1 Write neat code by selecting appropriate data structure and demonstrate a working solution for a given problem.
- 2 Demonstrate the ability to write reusable code and abstract data types.
- 3 Demonstrate the ability to implement different data structures with variety of implementations.
- 4 Analyze and compare given algorithms for time and space complexity.
- 5 Handle all possible cases in designing an algorithm.
- 6 Demonstrate the ability to write modular and re-usable code.

Course Contents

Introduction: Concept of Data types and Abstract Data types; Characteristics of an algorithm; Analyzing programs; Frequency count; Time and space complexity; Big 'O' and 'Ω' notation; Best, average and worst cases; Programming language provided data types, operations on various data types; Dangling pointers and garbage memory

[4 Hrs]

Arrays, Searching and Sorting: Searching: linear and binary search algorithm; Hashing: hashing functions, chaining, overflow handling with and without chaining, open addressing: linear, quadratic probing; Sorting: bubble sort, selection sort, quick sort, merge sort, insertion sort. Time complexity analysis of searching and sorting techniques.

[8 Hrs]

Files and File Handling: Files handling: various library functions for handling files; system call interface and library interface; Different file formats like csv, pdf, odt, etc; Text and binary files; Programs to copy, concatenate, rename files; Programs to handle existing file types; arguments to main() ;

[6 Hrs]

Stacks and Queues: Stack and queue as ADT; Operations on stack and queue; Implementations using arrays and dynamic memory allocation; Application of stack for expression evaluation, expression conversion; Recursion and stacks; Problems like maze and knight's tour.

[6 Hrs]

Linked Lists: Dynamic memory management; List as ADT; Concept of linked organization of data against linked list; Singly linked list, doubly linked list, circular linked list; Representation & manipulations of polynomials/sets using linked lists;

[8 Hrs]

Trees: Basic terminology; Binary trees and its representation; Types of Binary tree; Binary tree traversals and various operations; Insertion and deletion of nodes in binary search tree; Applications of trees;

[8 Hrs]

Self Study Unit:

Introduction to graph data structure; Types of graphs: directed, undirected, weighted, unweighted
Graph representation: adjacency matrix, adjacency list; Graph traversal algorithms: depth-first search (DFS), breadth-first search (BFS);

Minimum spanning tree algorithms: Prim's algorithm, Kruskal's algorithm

[8 Hrs]

Text Books

- E. Horowitz, S. Sahni, S. Anderson-freed, "Fundamentals of Data Structures in C", Second Edition, University Press, ISBN 978-81-7371-605-8
- B. Kernighan, D. Ritchie, "The C Programming Language", Prentice Hall of India, Second Edition, ISBN 81-203-0596-5
- Y. Langsam, M. Augenstein and A. Tannenbaum, "Data Structures using C", Pearson Education Asia, First Edition, 2002, ISBN 978-81-317-0229-1

Reference Books

- Ellis Horowitz, S. Sahni, D. Mehta "Fundamentals of Data Structures in C++", Galgotia Book Source, New Delhi 1995 ISBN 16782928
- Jean-Paul Tremblay, Paul. G. Soresan, "An introduction to data structures with Applications", Tata Mc-Graw Hill International Editions, 2nd edition 1984, ISBN-0-07-462471-7